



ROAD CONDITION MONITORING SYSTEM

Case Study Intelligent Systems (WS22/23)
Final report

Group members

- Anmol Sharma
- Elham Alfuqara
- Tobias d'Apolonia
- Wen-Hao Wu

Table of Contents

1 Abstract	2
2 Introduction	3
Mobile application	5
2.1 Framework	9
2.2 Bulldozer	9
3 Cloud	12
3.1 MySQL	14
3.2 Database Structure	14
4 Dataset	16
5 The controller dashboard	18
6 The Video Masking process	22
6.1 Implementation	23
7 Severity classification	25
8 AI Improvements	26
9 Report generation	28
9.1 Motivation	28
9.2 Main Features	29
9.3 Methodologies	29
9.4 Survey results	31
9.5 SWOT analysis	33
10 Future work	34
Severity classification	34
Damage Heatmap	34
Maintenance prediction	35
11 Result & Conclusion	36
12 References	37
13 Appendix	38
13.1 Bulldozer file	38

13.2 Timetable	38
13.3 Sample of report	38

1 Abstract

Road damage monitoring is an essential process for maintaining the safety and functionality of transportation networks. Road damage can occur due to several factors such as weather, vehicle traffic, and natural disasters. When left unaddressed, road damage can lead to costly repairs and pose a risk to motorists. To address these issues, road damage monitoring systems can be implemented. These systems utilize sensors and cameras to continuously monitor the condition of the road surface and identify any potential damage. The collected data is then analyzed and used to prioritize repairs and maintenance activities. By regularly monitoring and addressing road damage, the road damage monitoring system helps to reduce the likelihood of accidents and costly repairs, ensuring the safety and efficiency of the transportation network.

Mobile applications can be a useful tool for road damage monitoring systems. They can be used to allow individuals to report road damages, track them and automatically collect and analyze the road damages via live mode quickly and easily. Hence the idea of building a monitoring system consisting of AI (Artificial Intelligence), mobile application, cloud, and controller dashboard.

Combining AI power with mobile application enhances the application functionality, improves the user experience, revenue streams, and cost savings. Moreover, this will bring the case study a step forward to be converted into a commercial system.

2 Introduction

Road damages, such as potholes and cracks, can have grave consequences for drivers, pedestrians, and the overall transportation system. These damages can create safety hazards, cause damage to vehicles, and lead to traffic delays. In addition, road damages can have an economic impact by making it more difficult for goods to be transported and for people to get to work or other destinations.

One of the biggest challenges in addressing road damages is the time and resources required to identify and repair them. Traditional methods of road damage detection, such as visual inspections by maintenance crews, can be slow and labor-intensive. In addition, these methods may not always be effective at identifying all types of road damages.

To address these challenges, many transportation agencies are turning to artificial intelligence (AI) to improve the efficiency and effectiveness of road damage detection. By using AI algorithms and machine learning techniques, it is possible to automatically detect and classify several types of road damages with a high degree of accuracy. This can help to reduce the time and resources needed to identify the road damages and can improve road safety and vehicle reliability.

Overall, the use of AI in road damage detection and repair is a promising approach that has the potential to significantly improve the safety and efficiency of transportation systems.

This project took place over two semesters. The first semester goal was to build a system that detects road damages using a GoPro camera and generate a spatio-temporal report containing the exposure damages found. While the second semester target was transferring the first case study into a commercial product besides enhancing the AI performance.

The second case study vision was to create a comprehensive road monitoring system, that links the proper authorities for road repairing with the population. The chart below illustrates the project motivation and the team brainstorming that lead to project scope.

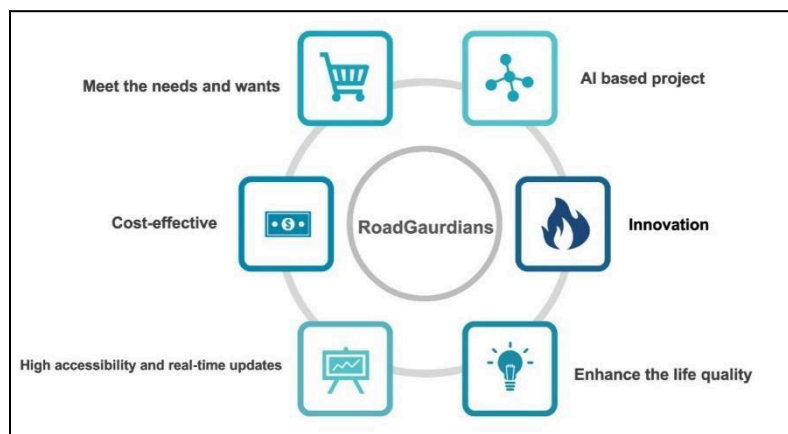


Figure 1 Motivation

Inspiring by the project vision, the project scope is defined as below:

“Build a user-friendly road damage monitoring system that consists of 3 main components: mobile application, cloud, and controller dashboard. Using AI-based model to recognize and classify road damages from user-collected-data. With a dashboard, the controller can easily access and manipulate the road damage data. In addition, the user should be able to create a report based on filters.”

Subsequently, the system consists of mobile application, cloud, AI and controller dashboard. Figure 1 illustrates the systems parts.

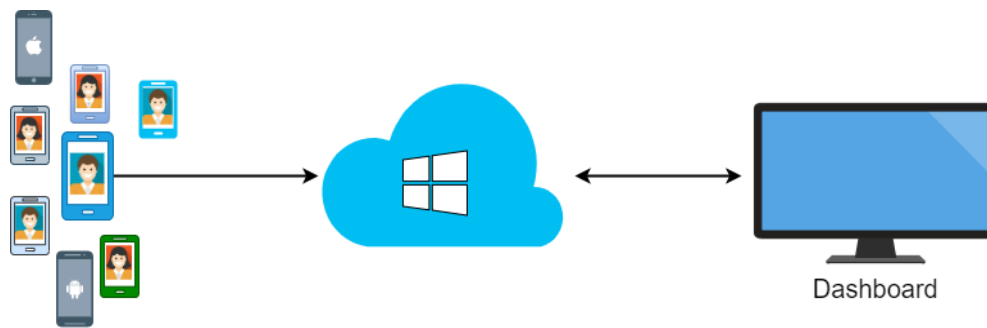


Figure 2 Project schematic

The app would allow users to take a photo of the damage which would then be sent to the proper authorities for repair. The app could also include a map feature that shows the location of reported road damage, allowing users to activate the notification system. Additionally, the app could notify users when a reported issue has been fixed. Overall, the goal of the app would be to make it easier for people to report road damage and for authorities to track and fix issues in a prompt and efficient manner, ultimately leading to safer and better-maintained roads for everyone.

A mobile application for road damage detection can improve the efficiency and effectiveness of road maintenance efforts, by allowing for real-time data collection, wider coverage, crowdsourcing, easy to use, and real-time mapping.

Linking the mobile application with Azure Cloud can greatly enhance its capabilities and benefits for the system. And by using Azure MySQL, we can ensure that the data collected by the application is stored in a secure, scalable, and high-availability database that can support the application's growth. The integration with other Azure services can also enhance the capabilities of the application and make it more cost-effective and efficient.

Furthermore, A controller dashboard is a critical part of a road damage detection system. It allows municipalities and private companies to monitor and manage the data collected by the system in real-time. The controller dashboard allows them to view the registered damages, change the status of repair, review the complaints, and take actions accordingly.

Moreover, the dashboard allows the controller to upload multi-recorded videos for damage detection and generate a comprehensive report.

However, the project execution phases are explained in detail in this report.

Mobile application

With the help of the community of users, the proper authorities will be informed about the road damages via user-friendly application that makes it easy for anyone to quickly and easily report potholes, cracks, and other damages.

Using the mobile application in this system has many advantages, such as:

- Speed up the repair and enhance road safety by providing real-time monitoring.
- Improve the accuracy of road damages detection and avoid subjective reports
- Cost-effective
- Increase the citizen engagement

The application was called **RoadGaurdian**, the inspiration behind the name is rooted in the idea of empowering individuals to take an active role in keeping our roads safe.

The application contains 8 main screens:

- Start screen: the user can choose to either Sign-in or Sign-up
- Sign-up screen: the user will be able to create an account by inserting his/her information and after agreeing to the terms and policy.
- Sign-in screen: the user will access his/her account using username and password.
- Main screen: the user can navigate between the different screens and functions like take pictures, live mode, maps overview, registered damages, and complain screen.
- Take picture screen: The user could take a picture of the road damage then send it to the proper authorities.
- Complain screen: through this screen the user can register a complaint and review the previously registered complaint.

The pictures below show the screens.



Figure 3 Start screen

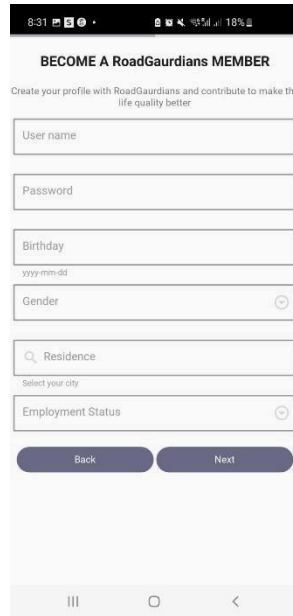


Figure 4 Sign Up screen

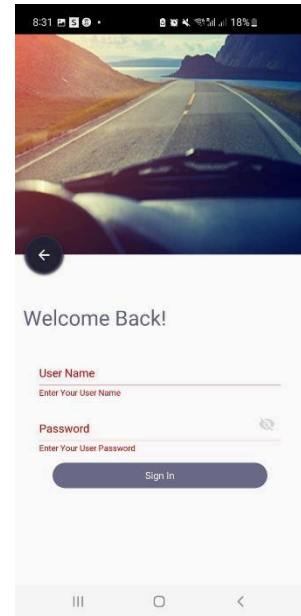


Figure 5 Sign In screen

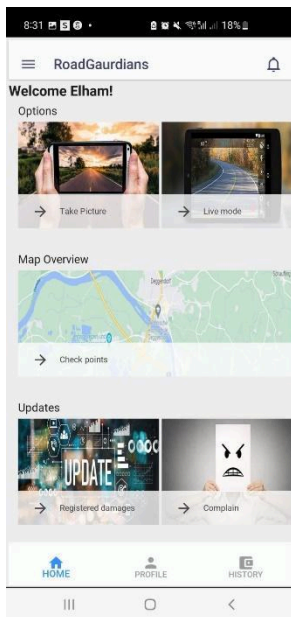


Figure 6 Main Screen

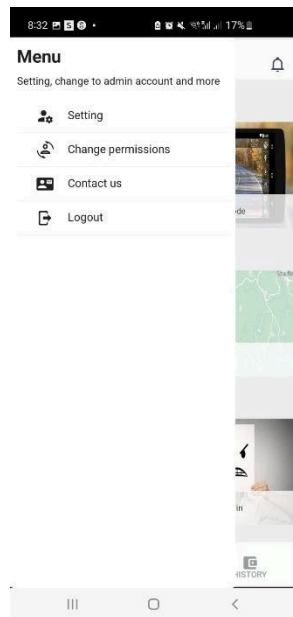


Figure 7 Main screen navigator



Figure 8 Take image screen

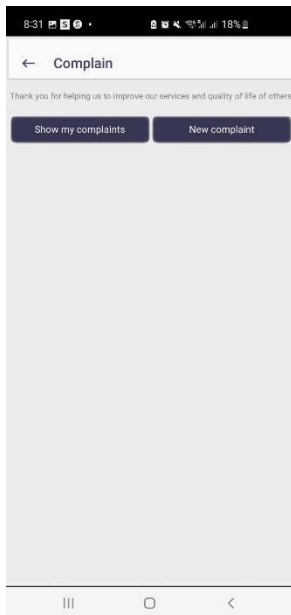


Figure 9 Complain screen

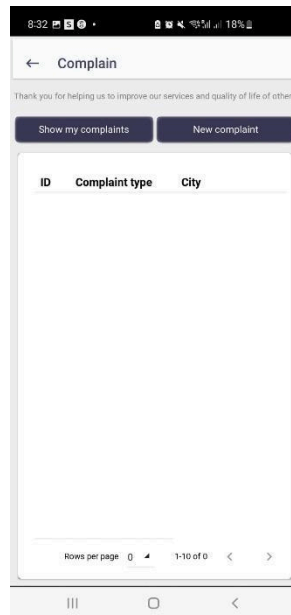


Figure 10 After selecting Show my complaints

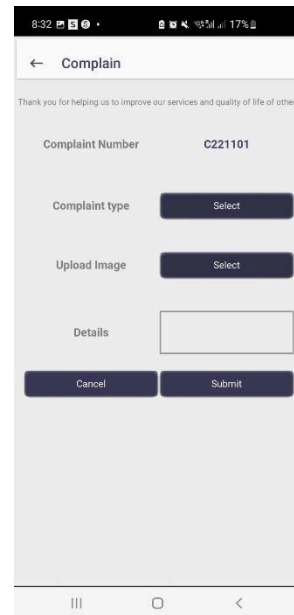


Figure 11 New complaint screen

Unfortunately, the connection between Android app and Azure MySQL still not established. For this iteration, the functions related to MySQL is not working on the real application, nevertheless, it working probably in the VScode. The mobile application still in the development stage and many features and functions could be added such as:

- User's profile
- Points systems
- Live mode
- Regular questioners to enhance the provided services

2.1 Framework

Kivy and KivyMD are open-source python libraries that can be used to create mobile applications. Kivy is a framework for creating multi-touch applications that run on a variety of platforms, such as Android and iOS. It is designed for creating high-performance, responsive, and cross-platform applications. KivyMD is a set of widgets for Kivy that provides material design elements, making it easy to create visually appealing and user-friendly applications.

For application coding, the code was divided into two main files, main2.kv file and main.py file. In Kivy, main2.kv file (short for Kivy Language) is used to define the layout and styling of the user interface for an application. This file uses a special markup language that is like HTML and CSS, but with some additional features that are specific to Kivy.

However, the main.py file (short for Python file) is used to define the behavior and logic of the application. This file contains Python code that is executed by the application.

It's important to note that while Kivy and KivyMD are powerful tools for creating mobile applications, they may have some limitations in terms of performance and compatibility with certain platforms. And it's also important to have a good understanding of Python and the libraries in order to use them effectively.

Once the application is complete, you can use tools such as Buildozer or PyInstaller to package the application for distribution on different platforms by generating APK files. In this project, Buildozer was used.

2.2 Buildozer

Buildozer is an open-source tool for packaging Python applications for distribution on mobile platforms such as Android and iOS easily and without the need for extensive knowledge of the underlying operating systems. This makes it easy for developers to create and distribute their applications to a wide audience.

However, there are some challenges when using Buildozer and KivyMD. One of the main challenges is that Buildozer requires a lot of dependencies to be installed on the development machine, such as the Android SDK and NDK, which can be a time-consuming and complex process. Additionally, Buildozer is not always fully compatible with the latest version of Kivy and KivyMD, which can cause some issues when developing the application.

Through this project, converting the desktop application into APK file was one of the major challenges. For that reason, the best practices to deploy the python code into Android application using Windows 10 are mentioned below:

1. Install Windows Subsystem for Linux (WSL)
 - a. Run Windows PowerShell as administrator
 - b. Then navigate to the official Microsoft Installation instructions which could be found <https://learn.microsoft.com/en-us/windows/wsl/install-manual>
 - c. After step 3 you should restart your computer
2. Install regular Ubuntu app from Microsoft Store.
3. Make sure that all application files are in one folder.
4. Make sure that all the used images are 520x520 png images.
5. Open Ubuntu app and select your username and password.
6. Then navigate to the same directory where the project/application saved
 - a. The way to access the windows files system through ubuntu terminal is slightly different than what used to, instead of using "`cd Users/../../`", (`cd /mnt/c/Users/.....`)
7. Install Buildozer by navigating through the official Buildozer documentation which could be found <https://buildozer.readthedocs.io/en/latest/installation.html>
 - a. Before starting, make sure pip3 is install, if not just run the command "`sudo apt install python3-pip`"
 - b. After installing all the Buildozer dependencies,
 - c. Then use the command "`buildozer init`", which will create a buildozer file on the project directory.
 - d. Change the buildozer contents based on the project requirement.
 - e. Run the command "`buildozer -v android debug`"
 - f. Most probably that the first error will show up and it will be an issue with the permission of the operation, if so, use this command "`sudo mount -t drvfs C: /mnt/c/o metadata`"
 - g. Wait till finishing the building process, and the apk file could be found in the project folder.

For more information, refer to Appendix 1 for the buildozer file.

After building the application and transferring it to Android mobile devices, it is expected that the application will not work properly, and for debugging Android Studio is used.

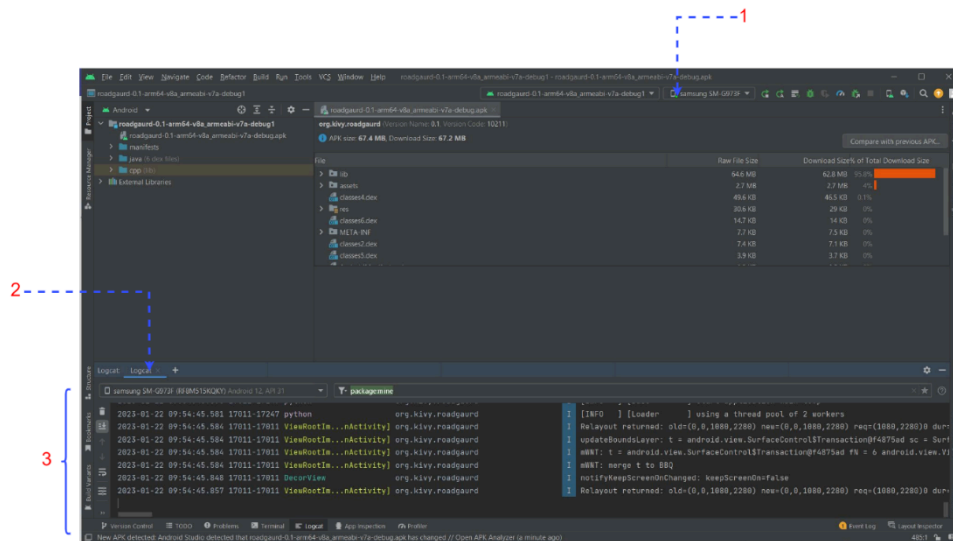


Figure 12 Android Studio main debug screen

The above picture shows the main window of Android studio. Android Studio is the official Integrated Development Environment (IDE) for Android development and it's an essential tool for debugging android applications. It provides a variety of features that can be used to identify and troubleshoot issues in your code, including a built-in debugger, breakpoints, and watchpoints.

Additionally, Android Studio provides a Logcat window (2), which displays log messages generated by the application (3). This can be useful for identifying issues that might not be obvious when running the application, such as errors that occur when the application is running on a device.

3 Cloud

Cloud computing is a type of computing that involves delivering computing resources, such as servers, storage, and applications, over the internet. Instead of having to maintain and manage these resources on-premises, businesses and individuals can access these resources as a service, paying only for what they need, when they need it.

When choosing the cloud options for the project, it's important to consider the features and services offered by the different providers and how they align with the specific needs of the project. One popular option is Azure, offered by Microsoft. Azure offers a wide range of services that can be used to build, deploy, and manage applications and services, including virtualized computing resources, storage, databases, and networking.

Azure has several benefits that make it an appealing option for many projects. One of the main benefits is its scalability, which allows businesses to scale their computing resources as needed and pay only for what they use. This helps to keep costs under control and ensures that the application can handle large amounts of data. Azure also provides a high level of security for data and applications, with features such as encryption, authentication, and access control. Additionally, Azure also provides ML service to build, deploy, and manages machine learning models. It provides a variety of tools for building and deploying machine learning models, including a drag-and-drop interface for creating machine learning workflows, and a variety of pre-built models that can be used to add intelligent features to applications.

Another advantage of Azure is that it offers a student account option, which allows students to use a limited set of Azure services for free. This is a great opportunity to gain hands-on experience with Azure and to learn how to use its services in real-world scenarios.

In addition, one of the key services that can be used in combination with Azure is Azure Database for MySQL. Azure Database for MySQL is a fully managed MySQL database service that is built on the Azure cloud platform. It provides high availability, scalability, and security for the applications. MySQL is used to store and manage the data collected from the application, such as users' data, registered complaints and registered damages.

To create MySQL database on Azure:

1. Navigate to Azure website.

2. Create an account, for this project students account was used.
3. Create a resource group
4. Create Azure Database for MySQL flexible server

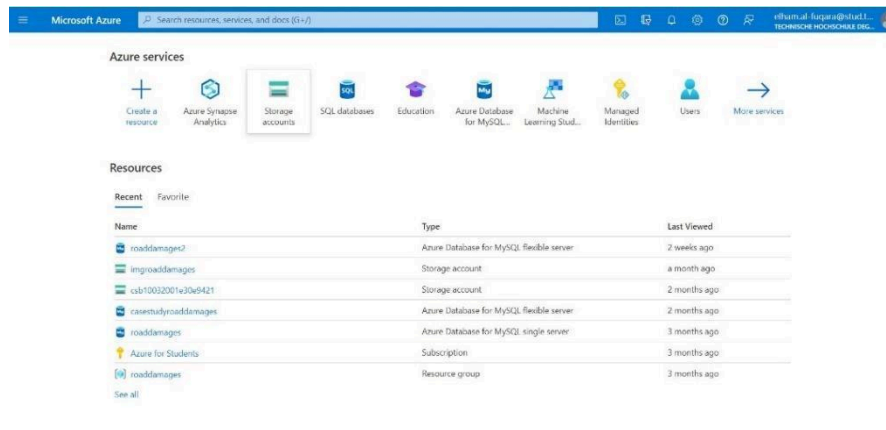


Figure 13 Azure main dashboard

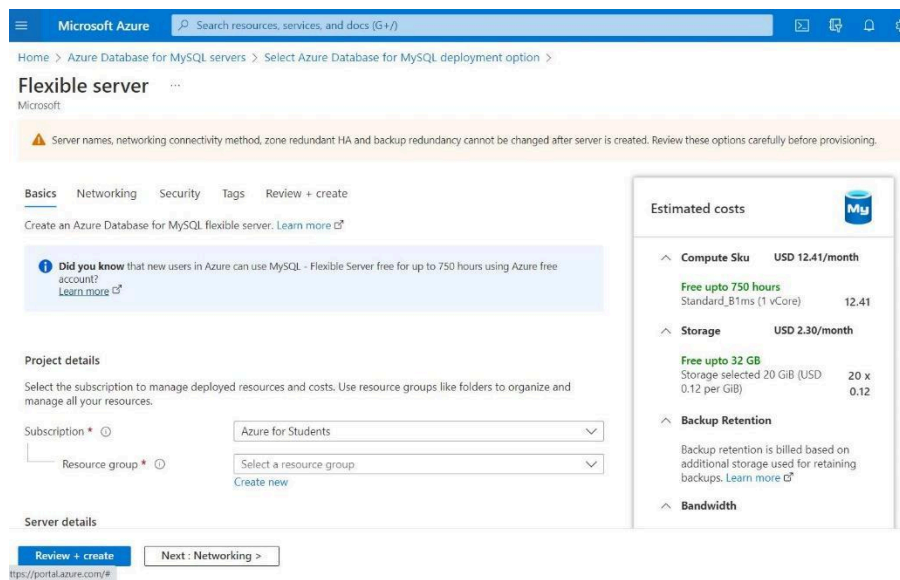


Figure 14 Azure Flexible server

3.1 MySQL

MySQL is a popular open-source relational database management system that is widely used for web and mobile application development. It stores data in tables, which consist of rows and columns, like a spreadsheet. It uses a Structured Query Language to interact with the data, allowing it to easily create, read, update, and delete data in the tables.

In this project MySQL workbench was used since it provides a visual interface for managing databases, performing database design, and running SQL queries.

To access the database using workbench, all you need is:

1. Install the workbench.
2. Setup a new connection using the server information as below:
 - Connection name: database name
 - Host name: Server name
 - Port: 3306
 - Username: Server admin login name
 - Password: Server password

3.2 Database Structure

We created a database called “roaddamages” on MYSQL workbench 8.CE. In this database we have created four tables for complaints, registered damages, users and points. As the name suggests, in the complaint table we collect all the registered complaints from the users. This data is arranged in 9 columns. In the table registered damages, the user can see all the registered road damages. In the table points users can see their gained points Similarly in the table users we see the details of the registered users. All the table consist of a primary key and a foreign key.

A primary key is the column or columns that contain values that uniquely identify each row in a table. A database table must have a primary key for Optim to insert, update, restore, or delete data from a database table.

A foreign key is a column or columns of data in one table that refers to the unique data values -- often the primary key data -- in another table. Foreign keys link together two or more tables. Figure below shows the database “roaddamages” on MYSQL workbench 8.CE.

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'complaints', 'points', and 'registered_damages'. The main area shows a query window with the query 'select * from complaints'. Below the query, the 'Result Grid' displays a table with 16 rows and 10 columns: idcomplaints, user_id, complaint_type, city, street, timestamp, complaint_status, complaint_details, and complaint. The data shows various complaints from users in different cities, with details on their status and repair costs.

idcomplaints	user_id	complaint_type	city	street	timestamp	complaint_status	complaint_details	complaint
1	1	1	Nürnberg	Adam Strasse	2022-12-03 12:01:11	0	Not Repaired	55.00
2	1	1	cham	Forstamt	2022-12-03 12:01:11	1		55.00
3	9	0	cham	adam	2022-12-03 12:01:11	0	repaired	55.00
4	1	0	Cham	Peter Strasse	2022-12-03 12:01:11	1	Not Repaired	55.00
5	9	1	Nürnberg	Peter Strasse	2022-12-03 12:01:11	0	Not Repaired	55.00
6	9	0	Cham	Forstamt Strasse	2022-12-03 12:01:11	1	Not Repaired	55.00
7	0	0	Cham	Forstamt Strasse	2022-12-09 12:01:11	1	Repaired	55.00
8	0	1	Cham	Klein Strasse	2023-01-09 12:01:11	1	Repaired	55.00
9	2	1	München	Markt Strasse	2023-01-11 12:01:11	1	Not Repaired	55.00
10	3	1	München	wald Strasse	2023-01-01 12:01:11	1	Not Repaired	55.00
11	3	1	cham	wald Strasse	2023-01-01 12:01:11	0	Not Repaired	55.00
12	3	1	cham	wald Strasse	2022-12-31 12:01:11	1	Not Repaired	55.00
13	3	1	cham	Bad Strasse	2022-12-31 12:01:11	1	Not Repaired	55.00
14	3	1	cham	Further Strasse	2022-12-31 12:01:11	0	Not Repaired	55.00
15	3	1	Cham	Further Strasse	2022-12-31 12:01:11	1	Not Repaired	55.00
16	3	1	Cham	Klein Strasse	2022-12-21 17:01:11	1	Not Repaired	55.00

Figure 15 MySQL workbench

For this project, a relational database structure was chosen, because of the interconnections of the different tables and the higher accuracy due to the fixed database structure. The database structure can be seen in the picture below.

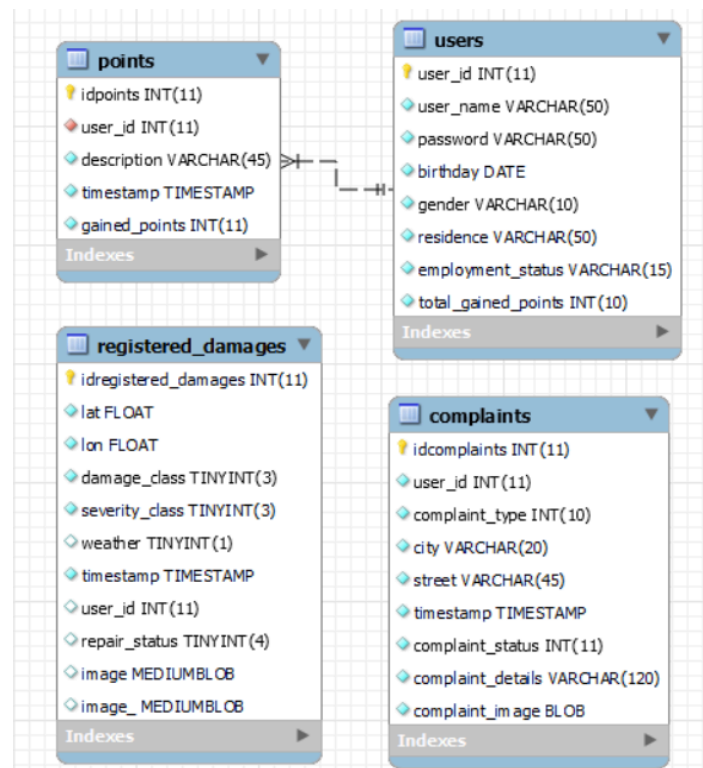


Figure 16 Database structure

4 Dataset

A high-quality road damage dataset is a critical component for training and evaluating machine learning models that can detect and classify road surface defects. When evaluating a road damage dataset, several criteria can be used to determine its quality and suitability for our project. In the following paragraphs, the key criteria of a high-quality dataset, the tools that were used, and some preprocessing steps that were applied will be discussed.

The tool we used is called Roboflow. Roboflow is a powerful and user-friendly tool for computer vision tasks. It is an end-to-end solution that allows users to easily collect, preprocess, and annotate images, making it an ideal choice for training and evaluating machine learning models. With Roboflow, we easily managed and organized our datasets, streamlined the annotation process, and quickly preprocessed our data. Roboflow also provides a wide range of features to customize the preprocessing and annotation process according to specific needs, which makes it suitable for increasing the damage instances. The only missing feature of Roboflow in our opinion is that if users could add preprocessing step to one specific class it would be perfect to increase instances of under-represented classes.

Completeness is one of the most important criteria for a good road damage dataset. The dataset should contain all the relevant data needed to achieve a good training result. This includes a wide range of road damage types, such as potholes, cracks, and other surface defects. However, this is more challenging than the collection of other datasets (such as human faces, car plates, etc.) since road damages do not have a clear uniformity and characteristics due to the nature of road damages. For example, when a linear crack is at a 45-degree angle, do we label it as longitudinal crack or as lateral crack? When there is an alligator crack combining with linear cracks, do we label the entirety as alligator crack or linear cracks and alligator crack? There are multiple instances of such that we tried to avoid as much as possible since we understood even small differences in how the dataset was labeled will affect the training outcome.

The dataset should also be free of missing values, and any missing data should be clearly indicated. The dataset we found online includes many road pictures, however, a lot of them were unlabeled. Unfortunately, in Roboflow, there is no such feature to remove unlabeled

pictures. In order to avoid having unlabeled pictures in our dataset, we wrote a python script to remove all the pictures without any label before we uploaded them to Roboflow.

Accuracy is another key criteria for a road damage dataset. The data should be accurate and free of errors, and any errors should be removed. This is especially important for machine learning models that rely on the dataset for training and evaluation. An accurate dataset ensures that the model will be able to accurately detect and classify road damages in real-world scenarios. However, this is also one of the most repetitive and time-consuming tasks. In our case, since the dataset we found from an open-source project was labeled by different people from different countries, there are labels that do not meet our expectations. In order to improve the accuracy and avoid “Garbage in, Garbage out”, we spent a lot of time on relabeling pictures and ensuring only the well-labeled pictures are added in the dataset.

Consistency is another important criteria for a good road damage dataset. The data should be consistent from the point of view/camera angle and should be formatted in a consistent manner. This ensures that the dataset is easily interpretable and usable for machine learning models. In our case, we ensured that all the pictures in the dataset are taken from the windshield of a car or at least simulate the angle from a car’s windshield. With this intentional ‘bias’ the model could learn and expect what it would be analyzing in a real world scenario, since the usage of our product is recommended to be used on a car windshield. In addition, all the pictures were resized in Roboflow to 640x640 to ensure uniformity and consistency.

In conclusion, a good road damage dataset should meet the criteria mentioned above. These criteria ensure that the dataset is representative of real-world road conditions, is accurate and of high-quality, and can be easily used and shared for training and evaluating machine learning models.

5 The controller dashboard

The controller dashboard consists of four different screens:

1. Log-In screen
2. Main screen
3. Analytics screen
4. Complains screen

1. Log-In screen

The Log in screen is the first screen shown, after the application is started. In a further version, this could be used to implement a user management system and ensure that only verified users can access the application.

2. Main screen

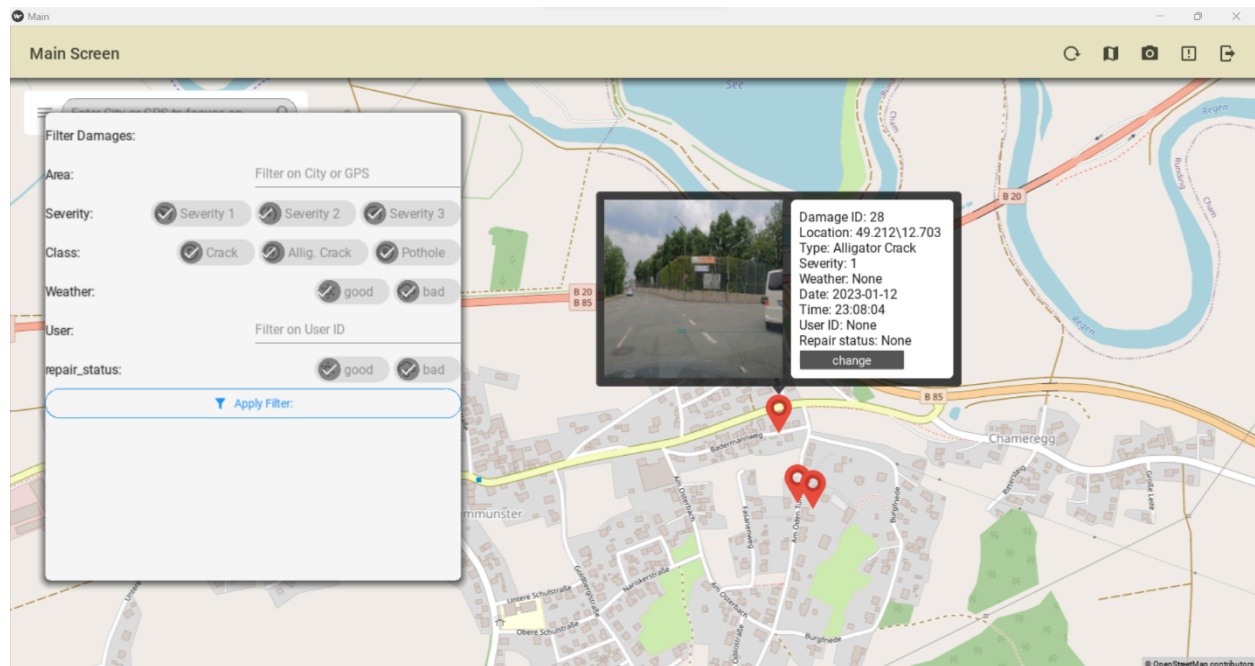


Figure 17 The main screen

The main screen is designed like google maps, to give users a known environment, what will make it easier to use. In this screen the user can access and interact with the damages registered in the database. Those are shown by markers like the markers in google maps. If the

users click on one of those markers a small box will pop up over the original marker. This box shows on the left side a picture of the damage. On the right side it will show information collected about the damage. This information holds the following attributes:

- The ID of the damage,
- GPS coordinates,
- The damage class,
- The severity of the damage,
- The weather condition,
- The ID of the user that registered the damage

Of those features most are already implemented in the current version of the application. For the other information further work is necessary. But to make this application more flexible for improvements most of the features are already implemented. Attributes of the damage, that cannot be registered until now will be assigned with the “None” value.

To make the navigation in the map easier, for the user we implemented a search button, that like in google maps guides the user to a certain location. Therefore, the user only needs to type in the coordinates or the name of the place he wants to navigate too. If the user click on the “hamburger menu” a new box will appear. This box has buttons of input boxes, that can be used to filter the damages shown on the screen. After the choice of the desired filters, the button apply filters needs to be pressed. During the filtration process the screen will freeze. Usually, this freeze should not be long, but if a city name is used to filter on this could be a lot longer. This is due to the network requests necessary to get the address of the damage based on the Coordinates. This process could be speeded up by doing this process in a different thread of the program as soon as the user selects the filter button.

In the top right corner of the main screen, there are 5 icons. Each of those icons has a tooltip function implemented. This means that if the user is with his mouse on one of those icons, without clicking it, he will see a small text explaining their function. The first of those buttons is the reload button. This can be used to reload all damages from the database. This is also the only button that will disappear as soon as the screen is switched. The second button that looks like a map can be used in a different screen to switch back to the main screen. The third button, that looks like a camera, can be used to switch to the analytics screen. The fourth button that looks like an exclamation mark can be used to switch to the complains screen and the last button can be used to log out. This will lead the user again to the log in screen.

3. Analytics screen



Figure 18 The analysis screen

In this screen the user can, like in the first version of this application, analyze the GO-PRO videos. But in comparison to the first version this screen is significantly improved. First it gives the user the option to select multiple videos. This can be done, by clicking in the “+” icons. If the users select a wrong file, that is not a video, this will not be added and shown on the screen. If the user wants to unselect a file, this can be done by clicking on the “-” icon. The list of videos is scrollable, what makes it possible to theoretically select an infinite number of videos. Underneath the choice of the videos the user can select with 3 checkboxes if he wants a report to be generated in word and or PDF format and if he wants his damages uploaded to a database. At the bottom of the screen is the start button, which will start detecting the videos. During the detection runtime, the screen doesn't freeze anymore, and the user gets information on the right side of the screen of the detection progress status. After the detection, the PDF and Word file are saved in the subfolder reports.

4. Complain screen

In the complaint screen the users can see all the registered complaints on the database in the form of a table. The table consists of columns such as complaint Id, user Id, city name, street name, type of damages, complaint type, date. All the registered complaints are shown by clicking the apply icon in the Apply filter section. The users can also filter out the registered complaints based on a filter such as a date, city, or complaint status in the Apply filter section. In the date filter user can choose the date from the range that user wants to see the complaints and the city filter can filter out the complaints based on the name of the city, similarly in the complaint status filter user can the click on the check box of solved and unsolved complaints. Figure below shows the complain screen:

☐	ID Complaints	UserID	City	Street	Complain Type	Complain
☐	3	9	cham	adam	0	0
☐	11	3	cham	wald Strasse	1	0
☐	14	3	cham	Further Strasse	1	0
☐	21	5	Cham	Kloster Strasse	1	0
☐	26	8	Cham	Ludwig Strasse	1	0
☐	27	9	Cham	Ludwig Strasse	1	0
☐	28	10	Cham	Ludwig Strasse	1	0

Figure 19 Complain screen

6 The Video Masking process

For the Video masking we tried several methods but ultimately used the AI model Detectron2 of Facebook. This object detection Model has 3 different operation modes:

1. Instance detection

In Instance detection the Objects are detected with bounding boxes, like it's shown on the picture to the left. For each instance of the pertained object, they will be also detected with a confidence level. This can be compared to most of the detection model like YOLO.



2. Semantic Segmentation

In Semantics Segmentation not only are the Bounding boxes detected, but the object itself is also detected. In the Picture on the left this is shown in an example with balloons. You can see in the picture, that around the balloons a line is drawn. This could be used in the future to improve damage detection and severity classification.



3. Panoptic Segmentation

In Panoptic Segmentation every Pixel of the Picture is assigned to a class. This is a major advantage and the reason we use this detection to mask our videos. Everything that is detected and not recognized as a road will be masked out.



6.1 Implementation

We implemented the detection model on the THD-GPU Server as a preprocess since this would slow down the detection too much. In the picture series below, is an example of the masking process shown.

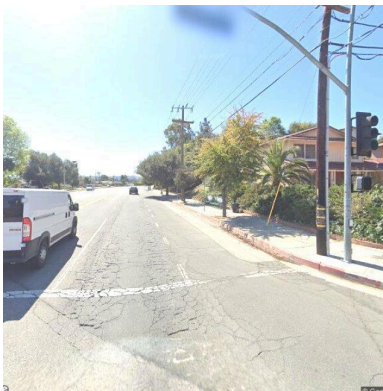


Figure 23 original picture



Figure 24 Segmented picture



Figure 25 Masked picture

The result of the detectron2 panoptic segmentation is a tensor with the shape of the picture. Each of the values in the tensor (each pixel) will be assigned to a class as a numerical value. An example of this is shown below.

```
Tensor ([[9, 9, 9, ..., 9, 9, 9],
        [9, 9, 9, ..., 9, 9, 9],
        [9, 9, 9, ..., 9, 9, 9],
        ...,
        [7, 7, 7, ..., 7, 7, 7],
        [7, 7, 7, ..., 7, 7, 7],
        [7, 7, 7, ..., 7, 7, 7]])
```

Each of those values can then be looked up to find out the corresponding class. In our case the relevant classes are 21 (road) and 44 (pavement) After the preprocessing of the tensor we will replace each instance of the values 21 and 44 with 255 and all the other classes with 0 to get our corresponding mask. If now we apply the bitwise and operation of the image and the mask all the pixels that correspond with our class will not be affected, but the color intensity of the other pixel values will be set to 0. Therefore they will be shown in black in the picture. We implemented that logic for three different use cases, first to test it on single pictures, second to videos and for the masking of pictures in a hole folder. The masking of image folders can be used to adjust the image for verification to improve the evaluation of the detection model and the video masking can be used to preprocess the videos before detection.

7 Severity classification

For the severity classification OpenCV was used. For each Damage that was found in a picture the bounding box will be used to mask out everything except for the damage. After this the Canny edge detection is applied to detect the edges in the masked picture. Following this the cv2.findContours Function is used. This will return the white area in the picture. This value can then be used and compared to a threshold depending on the class to detect the Severity of a damage. An example of this can be seen in the picture below.

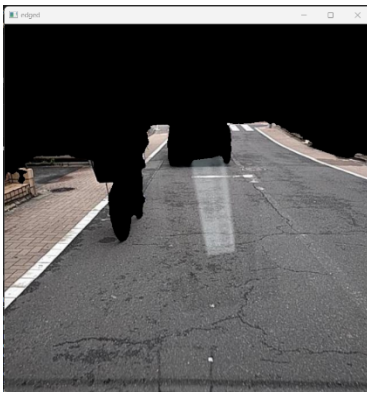


Figure 26 Original picture



Figure 27 Edge detection

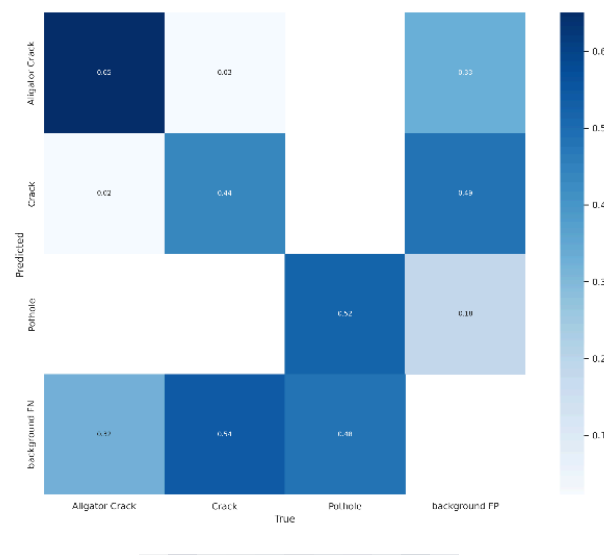


Figure 28 Contour detection

In this, there is still room for improvement, not also the damage, but other contours are detected too. But for a first estimation this is better than the results achieved with an AI detection or just using the bounding box size for a severity estimation.

8 AI Improvements

After training the YOLOV7 model, looking at the results of the confusion matrix, there seemed to be no improvement to the YOLOV5 model we used in the previous Semester. Since the detection percentage of the classes is not higher and the detection of noise is still quite high. But just looking at the confusion matrix doesn't show the full picture.



After looking at one of the examples batches below it is clear that almost all the damages are detected with a minimum confidence of 0.3. And due to the earlier masking, it is extremely unlikely that noise is detected. The issue why the parameters of the confusion matrix seem so poor can be seen in the picture below where the labeled and predicted pictures are compared. In the Top left picture, the person who labeled this image marked a lot of individual cracks, but the AI detected it as one big alligator crack. For the confusion matrix this means that the value calculated will be lower because the labeled cracks are not detected, even if detecting this as one alligator crack is not wrong. In Addition to this if we look at the second picture in row two, we can see that a crack was detected inside an alligator crack. This would lead to an increase in the noise parameter.

To conclude this, there is still some room for improvement on the confidence level of the damages but due to the masking we can reduce the confidence necessary to decide if a damage is a damage, which will lead to a better detection rate.

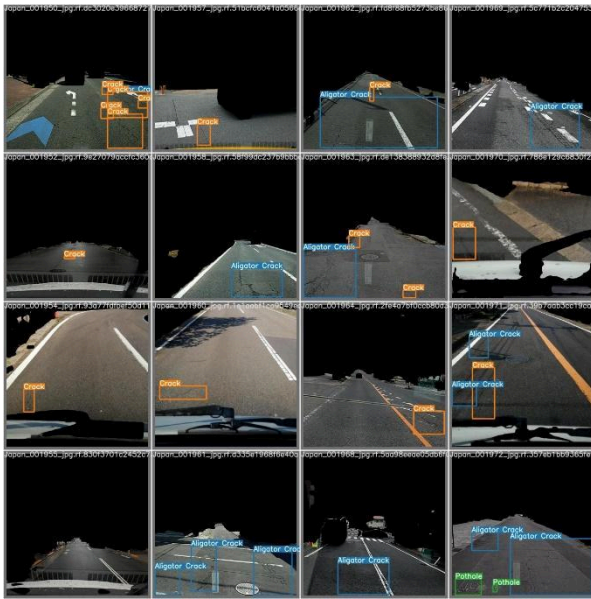


Figure 30 Labeled pictures

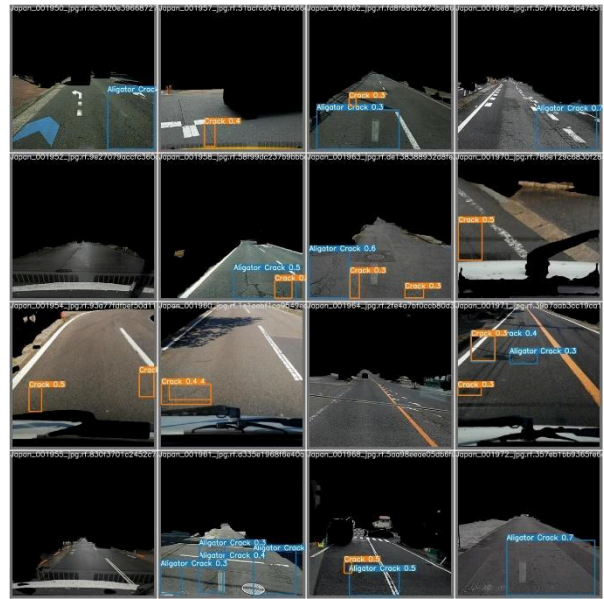


Figure 31 Detected pictures

9 Report generation

In this project, an automatic report generation tool has been developed. In the following chapter, the motivation to create this tool, the main features of the tool, the methods that were used to create this tool, and the problems faced while creating the report will be elaborated.

9.1 Motivation

During the last phase of the project (the previous semester), we already created a report generation tool. However, we were not totally satisfied with the product. The previous version of the report generation tool included the segmentation of the road, which is clearly stated in the project requirements that should be removed. And due to the tool we used for generating the drive route in the previous version, the report generation was not in a timely fashion. In addition, the code we implemented and the library we used were not optimal. Therefore, the motivation is clear to improve the report generation by generation speed and optimization of the code.

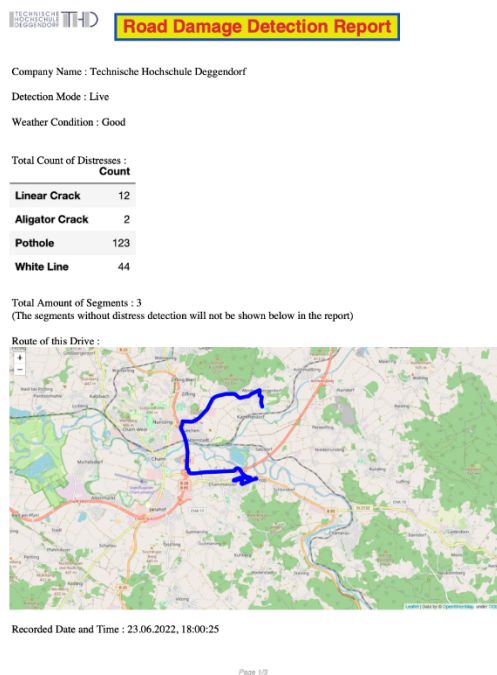


Figure 32 Previous version of report

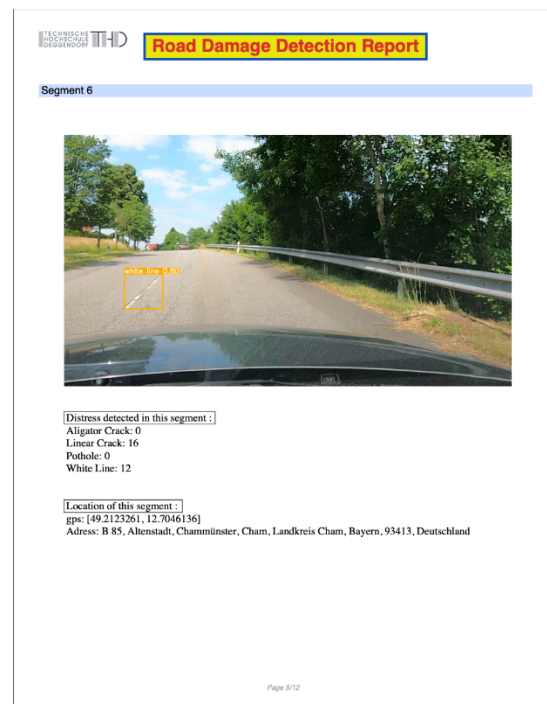


Figure 33 Previous version of report

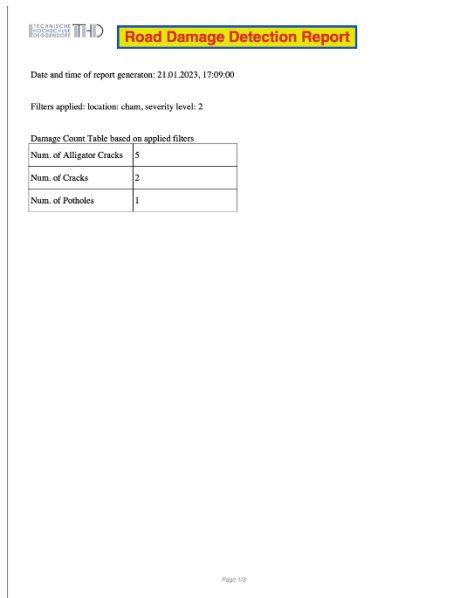


Figure 34 Current version of report

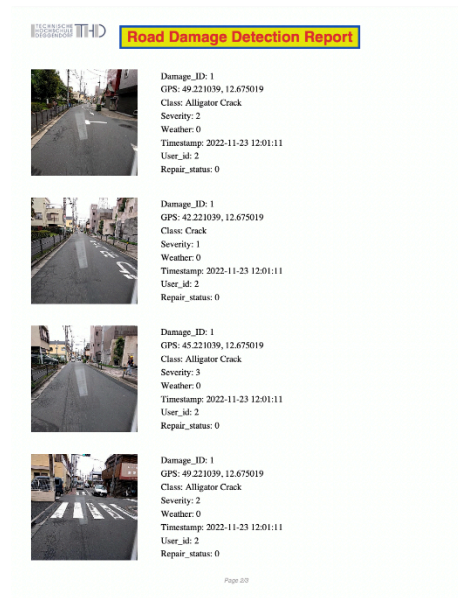


Figure 35 Current version of report

9.2 Main Features

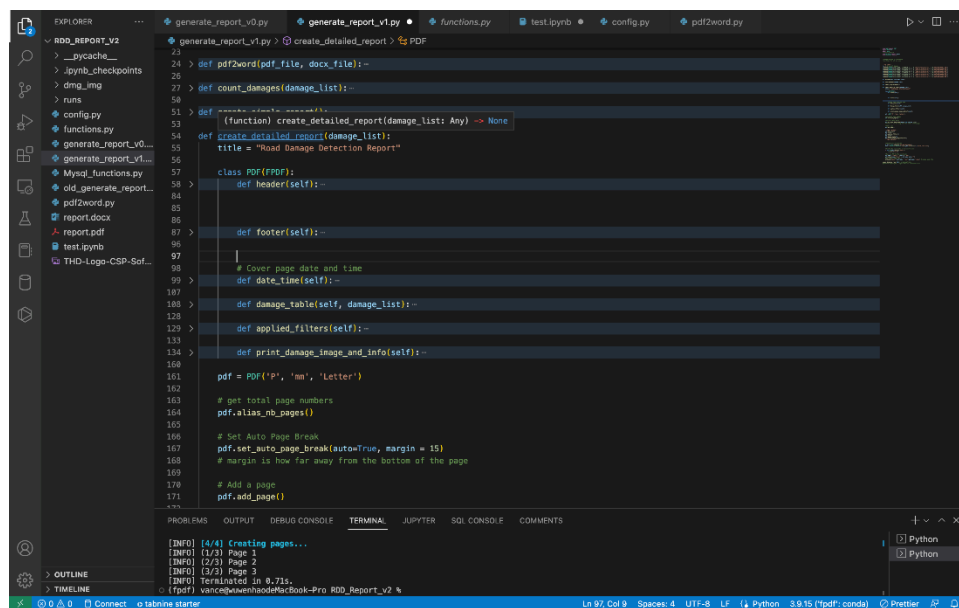
The main features of the report generation tool are as follows: Display of date and time of creation of report, a damage count table, display of detected damage, information of the detected damage (Damage_ID, GPS coordinates, Damage class, Severity, Weather, Timestamp, User_ID, repair status). The first page of the report is a cover page that contains a table of the overall damages. Every page of the report contains 4 damages with their details.

9.3 Methodologies

In this chapter, not only will the main methodologies be discussed but also the different methodologies used between the previous version of report generation and the current version will be compared. The report generation tool is based on Python and its libraries. The libraries used are as follows: FPDF2, datetime, pytz, pdf2docx. The main and most important library used is FPDF2. FPDF2 is an open-source library that provides simple and easy-to-use instructions for creating pdf documents. It allows developers to create documents with various page sizes, margins, and orientations, and to include text, images, and tables in the document. We used this library to construct the structure of the report such as the title of the report, footer that shows the page number, to show and resize road damage pictures, and so on. The datetime library was used to extract the current date and time. To make sure the correct time zone is used, the pytz library was used. With Pytz's help, we could specify the time zone of

central Europe. Pdf2docx is a very strong library that allows users to convert pdf files to docx files. Since it is clearly stated in the project requirement that the users should be able to easily edit the report, this feature is essential.

In the previous version, pandas library was used to construct a data frame. And the dataframe_image library was used to convert data frames into images. In the end, fpdf2 was used to show the picture of the table on the cover page. This is not optimal. In the current version, using an algorithm, a grid like structure was created with inserted with values. This is a huge improvement.



```

23
24 > def pdf2word(pdf_file, docx_file):
25
26
27 > def count_damages(damage_list):
28
29
30
31 > def create_detailed_report(damage_list: Any) -> None:
32
33
34 def create_detailed_report(damage_list):
35     title = "Road Damage Detection Report"
36
37
38     class PDF(FPDF):
39         def header(self):
40
41
42         def footer(self):
43
44
45         # Cover page date and time
46         def date_time(self):
47
48
49         def damage_table(self, damage_list):
50
51
52         def applied_filters(self):
53
54
55         def print_damage_image_and_info(self):
56
57
58     pdf = PDF('P', 'mm', 'Letter')
59
60
61     # get total page numbers
62     pdf.alias_nb_pages()
63
64     # Set Auto Page Break
65     pdf.set_auto_page_break(auto=True, margin = 15)
66     # margin is how far away from the bottom of the page
67
68     # Add a page
69     pdf.add_page()
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300

```

Figure 36 Overall code structure

In conclusion, the auto-generating report application has been made and improved from the previous version for the end user to realize what kind of damages are there, being able to locate where the damages are, and being able to see the detailed information of the report. Last but not least, both a pdf version and docx version of the report are available for the end user to use.

Survey Results and SWOT Analysis

Survey and SWOT analysis are important tools as they provide valuable information and insights that can be used to inform decision making and guide strategy. One of the biggest risks was making our application attractive and useful from the users' point of view. The survey helped to gain deeper understanding of the needs, opinions and preferences of users', while the SWOT analysis helped to identify internal and external factors that may impact the success of the project and develop strategies to address potential challenges and focus on the opportunities. Together, survey and SWOT analysis can provide a comprehensive view and help ensure its success.

9.4 Survey results

The survey was sent to THD-Campus Cham professors and students, and it consists of 6 questions:

1. Which features of a mobile app are most important to you?
 - a. Interacting with other app users
 - b. Gaining points and exchanging them with services or products
 - c. Using features to make your life easier (Google maps, DB, Banking apps ...etc)
2. Should government digital services be used more often?
 - a. Strongly disagree
 - b. Disagree
 - c. Neutral
 - d. Agree
 - e. Strongly agree
3. Would you rather take the initiative to report problems that have an impact on people's daily life?
 - a. Yes
 - b. No
 - c. Maybe
4. Would you use an application to communicate the complaint or problem to the concerned authorities?
 - a. Yes
 - b. No
5. Do you believe that digital damage reporting will have a positive impact in the future?

- a. Yes
 - b. No
 - c. Maybe
6. Besides reporting the road damages, what features do you think will be useful to add?
(Be our inspiration)

The results were:

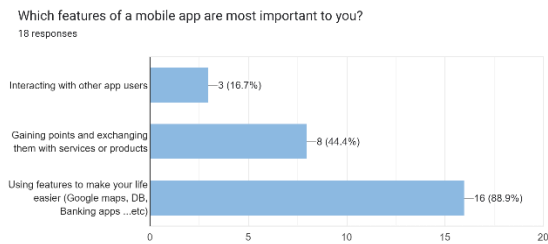


Figure 37 Question 1

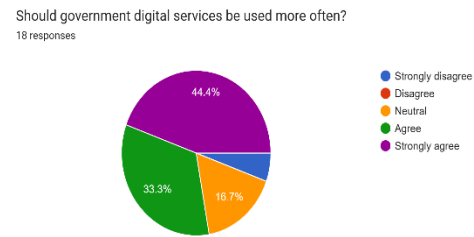


Figure 38 Question 2

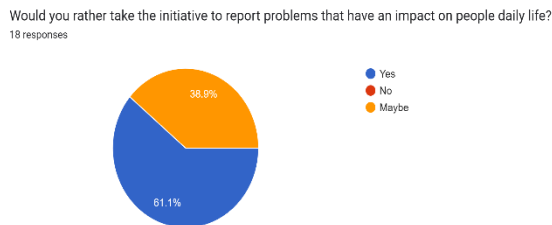


Figure 39 Question 3

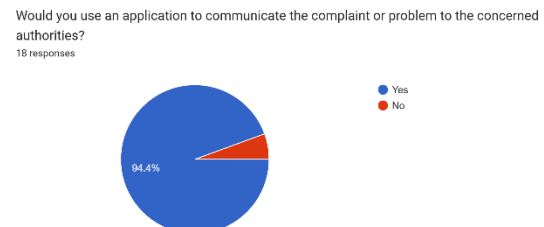


Figure 40 Question 4

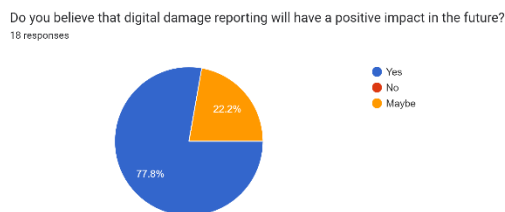


Figure 41 Question 5

The last question was the inspiration to create the complaint screen which has been shown in Application chapter, the survey participants suggest adding obstacles reporting, sidewalk damage reporting and traffic signs issues reporting.

9.5 SWOT analysis

The illustration below shows the SWOT analysis for this project:

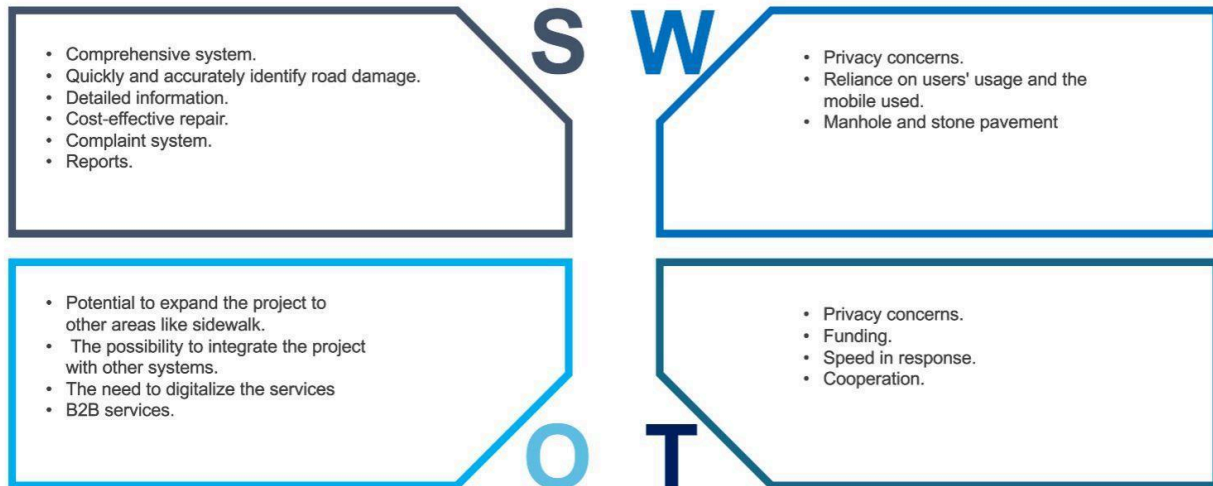


Figure 42 SWOT Analysis

10 Future work

For the future, the following points have the biggest potential for improvement:

1. Severity classification
2. Damage Heatmap
3. Maintenance prediction

Severity classification

As mentioned before, the classification with OpenCV has several disadvantages. To further improve this one option is to train an object Segmentation model for damage detection. This might be difficult for overall detection, because of the high effort for the dataset creation, but if the YOLO Model is used for detection the picture could be masked and the damage can be isolated. Like this a high detection rate can be achieved and it is very likely that the damage can be segmented and the size properly identified.

Damage Heatmap

If the damages are just plotted as markers on the map, it will be quite hard for users to identify the roads that require maintenance. This is why the implementation of a heatmap, like shown in the picture below, could help the person responsible to identify the roads the really require the most maintenance.



Figure 43 Damage Heatmap

Maintenance prediction

With the data that will be collected during the runtime of the program a new dataset could be created that shows how the severity of a damage develops over time. With this another AI model could be trained to predict what damage will occur in the future. Roads can be maintained on low traffic days, before the severity will be too high.

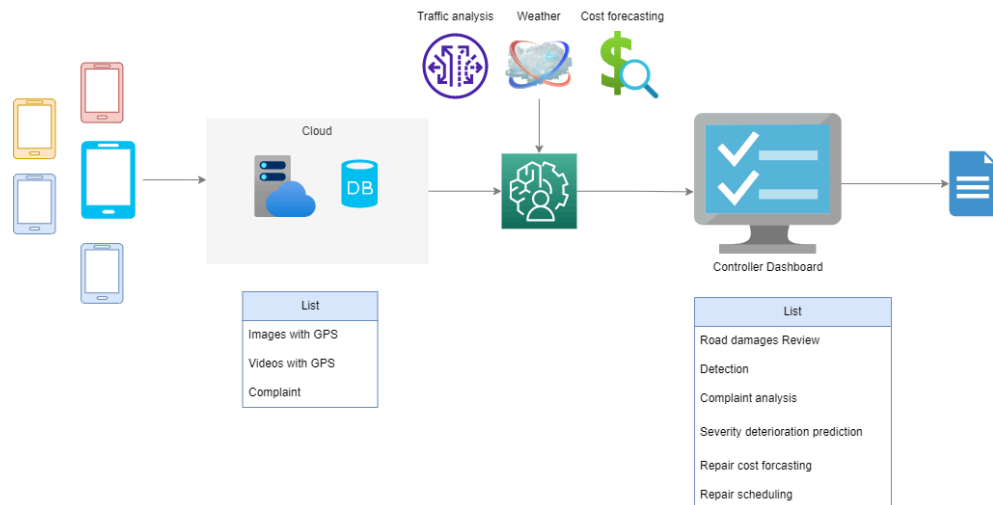


Figure 44 Comprehensive system

11 Result & Conclusion

Finally, it can be said that most of the goals of this project were achieved. The detection rate could be significantly improved due to the pre-process of masking our pictures as well as spending more time on labeling picture and cleaning the data. However, it is very time-consuming and unfortunately, we do not have enough time. Furthermore, an implementation of the severity classification was implemented

In addition to that, the user interface is a lot more interactive, more resilient and offers a wide range of new features for the user.

Also, the report generation was improved. Now instead of the segmentation approach that we used in the previous version, we consider every damage that has a certain difference in its gps location. The report can be now also generated in both pdf format and docx format.

Unfortunately, due to the limited time available some features needed to be neglected. However, they could be implemented as an isolated project or during a master thesis. First, a report creation based on the data available in the database. Second the connection problems of the application and the database. Third, a semantic segmentation of the damages to further improve the severity classification.

12 References

1. Seungbo Shim, Jin Kim, Seong-Won Lee, Gye-Chun Cho (2022). Road damage detection using super-resolution and semi-supervised learning with generative adversarial network. doi. 10.1016/j.autcon.2022.104139
2. Jongwoo Ha, Dongsoo Kim, Minsoo Kim (May,2022). Assessing severity of road cracks using deep learning-based segmentation and detection. doi.10.1007/s11227-022-04560-x
3. Zhang, Hongwei, Zhaohui Wu, Yuxuan Qiu, Xiangcheng Zhai, Zichen Wang, Peng Xu, Zhenzheng Liu, Xiantong Li, and Na Jiang. 2022. "A New Road Damage Detection Baseline with Attention Learning" doi.10.3390/app12157594
4. Patrick Liq Yee Tiong, Mushairry Mustaffar and Mohd Rosli Hainin (2012). Road surface assessment of pothole severity by close range digital photogrammetry method. doi. 10.5829/idosi.wasj.2012.19.06.3353
5. MYSQL 8.0 Reference Manual (2023).
6. MySQL 8.0 Reference Manual (20-01-2023). https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/index.html
7. KivyMD 1.1.1 documentation (2022). <https://kivymd.readthedocs.io/en/1.1.1/#>
8. <https://learn.microsoft.com/en-us/azure/?product=databases>
9. <https://roboflow.com/>
10. <https://learn.microsoft.com/en-us/windows/wsl/install>
11. <https://buildozer.readthedocs.io/en/latest/installation.html>

13 Appendix

13.1 Bulldozer file

13.2 Timetable

13.3 Sample of report